

MES

Gold Harvest MES Arayüz ve Altyapı

Operatörün ve müdürün gördüğü ekranlardan, bunları ayakta tutan üç servise: GitHub, Vercel, Supabase.

Kapsam

Operatör ekranı + müdür panosu

Altyapı

GitHub · Vercel · Supabase

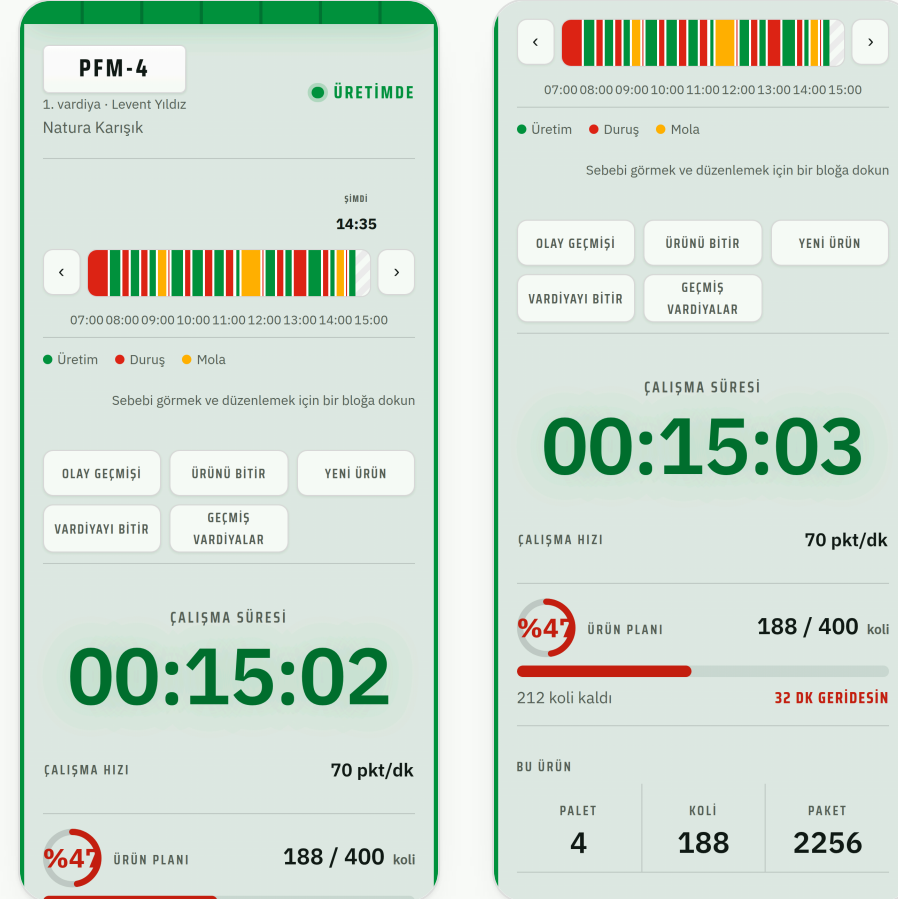
Hazırlayan

Burak Rıdvan Göker

Operatör ekranı

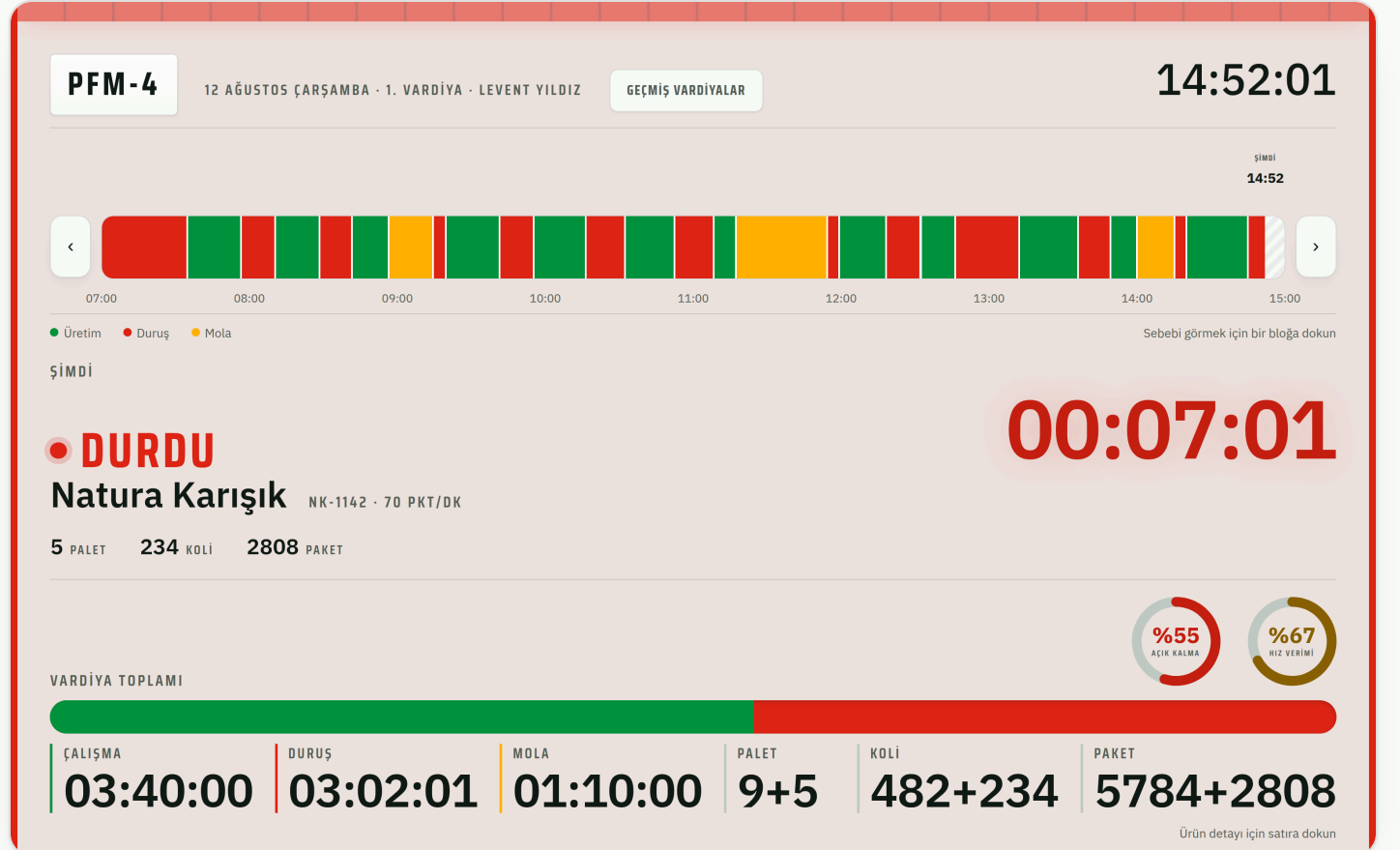
- Ekranın tamamı hattın durumuna göre renk alıyor — uzaktan bakan **tek renkte** durumu okuyor.
- **BAŞLAT · DURDUR · MOLA · +1 PALET** — dört büyük buton, eldivenle kullanılabilir.
- Duruş sebebi için kod ezberlemek yok; operatör ne olduysa yazıyor, sistem tekrarlayanları tek dokunuşluk seçeneğe çeviriyor.
- **Saat sonradan düzeltilebiliyor.** Olay, yaşandıktan dakikalar sonra girilse bile; düzeltince bütün süre hesapları kendiliğinden güncelleniyor.

Ekran görüntüleri örnek vardiya verisiyle alınmıştır.



Müdür panosu

- Hattın o anki durumu ve **ne kadar süredir** o durumda olduğu, ekranın en görünür yerinde.
- Vardiyanın tamamı **saat ekseninde**: hangi saatte ne olduğu tek bakışta; bir bloğa dokununca sebebi açılıyor.
- **Açık kalma oranı** ve **hız verimi** ayrı ayrı gösteriliyor — biri "durdu mu", diğeri "çalışırken yavaş mıydı" sorusunu cevaplıyor.
- Telefondan ve bilgisayardan, tarayıcıyla açılıyor; kurulum ya da uygulama indirme gerekmiyor.



Sistemi ayakta tutan üç servis

Üçü de hazır, dışarıdan kullanılan servisler — bu proje için sıfırdan sunucu kurulmadı.

GİTHUB

Kod deposu

Uygulamanın tüm kaynak kodu burada tutuluyor, kimin ne zaman değiştirdiğiyle birlikte.

VERCEL

Yayın

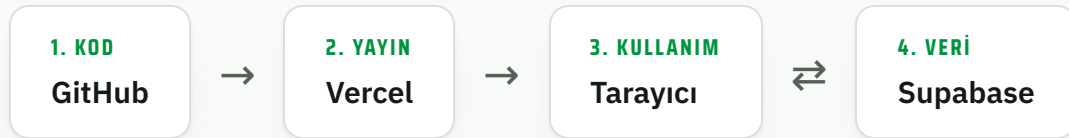
GitHub'daki koda her değişiklik geldiğinde, siteyi otomatik olarak yeniden derleyip yayınlıyor.

SUPABASE

Veritabanı

Vardiya, ürün, duruş, palet — tüm veri burada; bir değişiklik anında açık olan tüm ekranlara yansıyor.

BASİTÇE



Kod GitHub'da, tek bir depoda tutuluyor

- **Tek depo** (burakridvangoker/gold-harvest-mes) — projenin başından beri tüm kaynak kod ve değişiklik geçmişi burada.
- Değişiklikler doğrudan yayındaki dala değil, ayrı isimli "**dallar**" üzerinde yapılıyor (bu sunumun kendisi de bir dal üzerinde hazırlandı); iş hazır olunca yayın dalına aktarılıyor.
- Şu an yayına bağlı dal **main değil**, ayrı isimli bir geliştirme dalı — hangisi fiilen kullanılan/güncel dalsa "gerçek" kabul edilen odur, isim "main" olması onu otomatik öncelikli yapmıyor.
- Otomatik test/derleme hattı (CI, GitHub Actions) henüz kurulu değil — depoda böyle bir tanım yok; kontrol elle (`npm run lint` , `npm test`) ve Vercel'in kendi derleme adımıyla yapılıyor.
- Pull request akışı kullanılabilir ama zorunlu değil; küçük/acil düzeltmeler doğrudan ilgili dala da işlenebiliyor.

Veritabanı şeması da kodla birlikte, ama elle uygulanıyor

- Veritabanı değişiklikleri klasik bir "migration aracı" ile değil, düz **".sql" dosyalarıyla** yönetiliyor (ör. `setup_shifts.sql` , `add_mola.sql`) — depo kökünde, numaralı bir sırayla.
- Yazan tarafın (bu depoyla çalışan yapay zeka asistanı dahil) Supabase'e **doğrudan yazma yetkisi yok**: yeni bir .sql dosyası hazırlanır, tam içeriği burada gösterilir, ilgili kişi bunu Supabase SQL Editor'e elle yapıştırıp çalıştırır.
- Yaşanmış ders: yayın dalı bir süre **main'den aylarca geride** kaldı — main eski bir "an"da donmuştu, o anda vardiya modeli, zaman çizelgesi, hatta sayfa yönlendirme kuralı bile yoktu.
- Biri farkında olmadan main üzerinden yayın yapınca sitede aylar öncesinin arayüzü çıktı ve "her şey mahvoldu" izlenimi doğdu — oysa geliştirme dalına hiç dokunulmamıştı, sorun sadece yanlış daldan yayın yapılmasıydı.

main artık geliştirme dalıyla **bilerek eşit** tutuluyor — aksi halde bu karışıklık tekrar yaşanır.

Değişiklik push'landığı anda yayın süreci başlıyor

- Vercel projesi GitHub deposuna bağlı; "**Production Branch**" ayarı hangi dalın canlıya yansıyacağını belirliyor.
- Belirlenen dala her değişiklik gönderildiğinde Vercel otomatik bir derleme (`npm run build`) tetikliyor; birkaç dakika içinde yeni sürüm yayında — elle bir "yükleme" adımı yok.
- Aynı anda başka dallar için de ayrı, geçici **önizleme adresleri** üretilebiliyor; bunlar canlı siteyi hiç etkilemiyor.
- Yayın adresi tek; ayrıca bir sunucu/hosting kurup yönetmek gerekmiyor, bu işi Vercel üstleniyor.

Tek bir küçük dosya, iki büyük tuzak

VERCEL.JSON

```
{
  "rewrites": [{ "source": "/(.*)",
    "destination": "/index.html" }]
}
```

Uygulama tek sayfalık (/operator, /mudur gibi rotaları tarayıcı üretiyor); bu kural olmadan biri o adrese doğrudan girince sunucu "böyle bir dosya yok" deyip 404 verir.

- `VITE_SUPABASE_URL` ve `VITE_SUPABASE_ANON_KEY` Vercel proje ayarlarında ortam değişkeni olarak tanımlı; eksikse derleme sırasında hata verip site hiç açılmıyor.
- Vercel'de bir deployment elle de **"Promote to Production"** ile terfi ettirilebiliyor — bu yapıldığında ayarlar doğru görünse bile yayındaki sürüm yanlış olabiliyor.
- Belirtisi: canlı sitenin yüklediği `assets/index-*.js` dosya adı, yerel derleme çıktısındaki adla uyuşmuyor.

Tek veritabanı, sekiz ana tablo

line_status	hangi hat/makine kayıtlı	shifts	vardiyalar (operatör, hat, başlangıç)
product_runs	vardiya içindeki ürün üretimleri	timeline_events	üretim/duruş/mola durum geçişleri
pallet_records	çıkan paletler	stop_reason_segments	bir duruşun içindeki çakışan sebepler
personnel	personel listesi (ayrı sistemden kopya)	manager_dashboard_visits	müdür panosu ne zaman açıldı

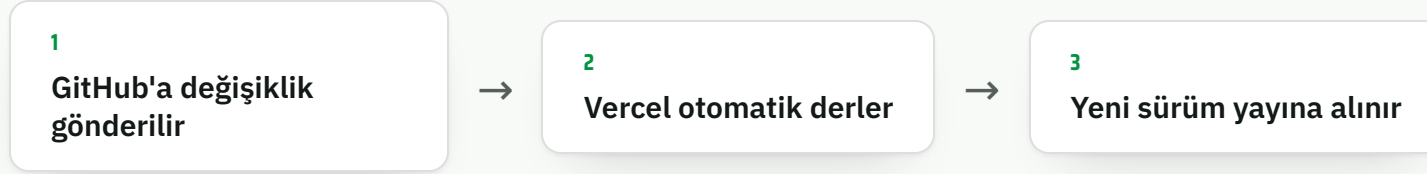
Tarayıcı, düşük yetkili bir **"anon key"** ile doğrudan Supabase'e bağlanıyor. Kimlik doğrulama henüz yok; tüm tablolarda Row Level Security açık ama politikalar `using (true)` — yani herkes okuyup yazabiliyor. Bu, sahada hızlı iterasyon için bilinçli bir tercih; bir kullanıcı/rol sistemi eklenince sıkılaştırılacak.

Süre hiçbir yerde saklanmıyor, hep hesaplanıyor

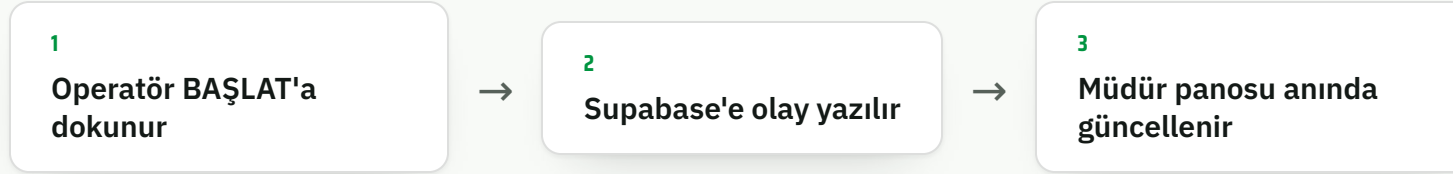
- Operatör bir olayı genelde birkaç dakika geç giriyor ("7:30'da makine durdu ama ben 7:35'te girdim") — bu sahada **istisna değil, kural**.
- Bu yüzden `timeline_events` bir durumun sadece **BAŞLADIĞI anı** tutuyor; süre bir sonraki olayın anına kadar hesaplanıyor, hiçbir "toplam süre" kolonunda saklanmıyor.
- Sonuç: geriye dönük saat düzeltmesi **yapısal olarak mümkün** — bir saat değişince komşu aralıklar otomatik kayıyor, boşluk ya da çakışma oluşmuyor.
- **Realtime:** Supabase'in canlı abonelik mekanizması kullanılıyor; bir kayıt değişince açık olan tüm ekranlara anında haber veriliyor.
- Kayıtlar sonradan düzenlenebilir/silinebilir olduğu için gelen sinyalde "artımlı yama" yapılmıyor, o vardiyanın **tamamı yeniden çekiliyor** — vardiya başına veri az olduğu için bu hem ucuz hem daha güvenli.

Bir değişiklik ekrana nasıl ulaşıyor

KOD TARAFI — BİR DEĞİŞİKLİK YAYINA ÇIKARKEN



KULLANIM TARAFI — OPERATÖR BİR TUŞA BASINCA



Bu iki akış birbirinden bağımsız çalışır: kod tarafı dakikalar süren bir derleme işi, kullanım tarafı ise saniyenin altında bir veri güncellemesi. Biri yavaşlasa da diğeri etkilenmez.

ÖZETLE

Üç servis, tek uygulama

KOD

GitHub'da, isimli dallar halinde, tüm değişiklik geçmişiyle birlikte.

YAYIN

Vercel'de; belirlenen dala her değişiklikte otomatik olarak yeniden yayınlanıyor.

VERİ

Supabase'de; gerçek zamanlı ve tek doğru kaynak — hiçbir rakam ayrıca saklanmıyor, hep ham kayıttan hesaplanıyor.

MALİYET

Üçü de hazır, dışarıdan kiralanan servis; bu proje için sıfırdan sunucu ya da altyapı kurulmadı.

Sorularını bekliyorum — istersen canlı üzerinde de gösterebilirim.